MusicCaps Pipeline - SFT 面试准备手册

本文面向本项目的 SFT（Supervised Fine-Tuning）实现，帮助你在面试中从「工程实现 + 方法论 + 取舍」三个层面讲清楚：

- 这套 SFT 在做什么
- 每个环节有哪些可选项
- 为什么这里这样选
- 替代方案的优缺点和适用场景

---

0. 先把最容易卡住的句子讲清楚

原句：**“只在 assistant 回复 token 上计算交叉熵损失。”**

0.1 assistant 是谁?

在对话格式里，一条样本通常有三段角色：

- `system`：系统规则（比如“只描述你听到的，不要编歌名”）
- `user`：用户输入（这里是音频 + 指令）
- `assistant`：模型应该给出的回答（训练标签是 MusicCaps 的 caption）

0.2 “回复”是什么?

就是 assistant 这段文本本身。
例如：

- system: 你是音乐听感助手
- user: 听音频并描述
- assistant: A mellow acoustic guitar melody with soft vocals...

这里最后一句就是“回复”。

0.3 token 在这里怎么定义?

模型不会直接处理“整句文字”，而是先把文字切成 token（子词单位）。
举例（示意，不是精确切分）：

- 文本：`acoustic guitar melody`
- token 可能是：`["acoustic", " guitar", " melody"]`

训练和损失都是在 token 级别算的，不是在句子级别直接算。

0.4 交叉熵是谁的预测和谁的标签?

对每个时间步 t：

- **预测方**：模型给出的“下一个 token 概率分布” $p_\theta(\cdot|x, y_{<t})$
- **标签方**：真实 token y_t（来自数据里的 caption）

token 级交叉熵（负对数似然）写成：

$$ \mathcal{L}_{\text{token}}(t) = -\log p_\theta(y_t \mid x, y_{<t}) $$

整段 assistant 的 loss 是这些 token loss 的平均或求和。

0.5 为什么是“只在 assistant 上算”？

因为我们真正要模型学的是“回答长什么样”。
system 和 user 是输入条件，不是学习目标。如果把它们也算进损失，模型会被鼓励“复读输入”，训练目标会变脏。

在实现里通过 `labels=-100` 屏蔽非目标 token（PyTorch CE 会忽略 -100）。

0.6 为什么用交叉熵，不用别的？

- 生成模型本质是在学条件概率分布 $p(y|x)$，交叉熵正好对应最大似然训练；
- 数学和工程都成熟，稳定、可微、实现标准化；
- 与推理时的自回归生成机制一致（训练-推理目标对齐）。

替代损失不是不能用，但大多用于特定场景：

- MSE：适合连续值回归，不适合离散 token 分类；
- hinge/ranking loss：适合排序任务，不直接建模完整文本分布；
- RL reward loss：适合后训练阶段做偏好优化，通常在 SFT 之后。

---

1. 全流程总览（先给面试官地图）

本项目 SFT 流程可以讲成 8 步：

1. 准备数据清单（manifest）
2. 构造对话格式（system/user/assistant）
3. 文本和音频打包成模型输入
4. 构造标签并做 mask（只保留 assistant）
5. 前向计算交叉熵损失
6. 反向传播更新 LoRA 参数
7. 训练稳定性控制（梯度裁剪、dtype、seed）
8. 保存 adapter 供后续 GRPO 使用

下面逐步展开，每一步都讲“定义-选项-取舍-属性”。

---

2. 数据清单（Manifest）

2.1 定义（是什么）

`manifest.jsonl` 是训练样本的一行一条记录，字段有：

- `wav_path`：音频路径
- `caption`：目标描述文本
- `aspects`：属性词（后续 GRPO 奖励会用）

- `split`：train/valid/test

2.2 功能（在项目里干什么）

它是“单一事实源”：

- 训练脚本不去猜文件名
- 数据问题尽早暴露
- 实验可复现（同一个 manifest 对应同一批样本）

2.3 二级选项

- **A. 显式 manifest（本项目）**
  - 优点：稳定、可审计、排错快
  - 缺点：前处理多一步
- **B. 运行时扫描目录自动匹配**
  - 优点：起步快
  - 缺点：隐式规则多，线上和离线可能不一致

2.4 替代方案

- Parquet/Arrow 数据格式
  - 优点：大规模 I/O 更快
  - 缺点：人工可读性差，开发复杂度更高

2.5 属性

- 复杂度：中
- 稳定性：高（显式数据定义）
- 成本：低到中
- 可解释性：高

---

3. 会话构造（system / user / assistant）

3.1 定义

我们把样本包装成聊天格式：

- system：行为规则
- user：音频 + 指令
- assistant：参考 caption（训练时作为目标）

3.2 功能

让模型在“真实推理形态”下训练，减少部署时格式偏差。

3.3 二级选项

- **A. 固定模板（本项目）**
  - 优点：实验可比性强，稳定
  - 缺点：表达风格可能单一
- **B. 多模板混训**
  - 优点：鲁棒性更好
  - 缺点：变量增加，定位问题更难

3.4 替代方案

- 指令自动改写增强（LLM augmentation）
  - 优点：覆盖更多说法
  - 缺点：可能引入噪声标签，损害稳定性

3.5 属性

- 复杂度：低到中
- 稳定性：固定模板高，多模板中
- 成本：低
- 可控性：高（固定模板）

---

4. Tokenization 与输入张量

4.1 定义

tokenization 是把“音频+文本对话”转换成模型可计算的张量（如 `input_ids`、`attention_mask`）。

4.2 功能

把人类可读输入映射到神经网络输入空间。

4.3 二级选项

- **A. 使用官方 processor（本项目）**
  - 优点：与模型预训练格式一致，兼容性最好
  - 缺点：对版本依赖更强
- **B. 自己拼输入**
  - 优点：灵活
  - 缺点：极易格式错位，出现隐蔽 bug

4.4 属性

- 复杂度：中
- 稳定性：官方 processor 高
- 成本：中（要管版本）
- 风险：手工拼接风险高

---

5. Label Masking（为什么只训练 assistant）

5.1 定义：什么叫「两次构造」？各是做什么用的?

人话版：同一条训练样本，用**同一套**对话模板（`apply_chat_template`），**各生成一份** token 序列。一份是「带标准答案的整段对话」，一份是「只有问题、没有答案」的前缀。第二份的唯一用途，是量出 **assistant 回复在 token 空间里从第几个位置开始**；然后在第一份上把这一段之前的 `labels` 全部屏蔽掉。

第一次构造：full（完整序列，含 assistant）

- **内容**：`system` + `user`（音频 + 指令）+ `assistant`（数据里的 `caption`）。
- **模板参数**：通常 `add_generation_prompt=False`，因为这是「已经有人类写好答案」的监督样本，不是「让模型接着往下生成」的推理态。
- **得到什么**：整条对话的 `input_ids`（以及 processor 附带的其它张量）。**前向传播、算 logits、最后算交叉熵，都建立在这条序列上。**
- **用途**：提供「条件 + 目标文本」的完整 token 串，作为训练时的输入与监督载体。

第二次构造：prompt-only（仅前缀，不含 assistant）

- **内容**：仍是 `system` + `user`，但**没有** assistant 那一轮；并配合 **`add_generation_prompt=True`**，表示「对话停在这里，接下来该模型生成」——与真实推理时「给到 prompt 再 generate」的形态对齐。
- **得到什么**：只有前缀的 `input_ids`，其序列长度记为 **`pl`**（prompt length，以 tokenizer + 模板为准）。
- **用途**：**不用于单独算 loss**。它只做一件事：告诉代码「在 full 序列里，前 `pl` 个 token 属于条件/prompt，从第 `pl+1` 个 token 起才是我们要监督生成的 assistant 内容」（边界由官方模板决定，而不是人工数字符）。

两次构造之后怎么 mask?

- 复制 full 的 `input_ids` 得到 `labels`。
- 把 `labels` 的**前 `pl` 个位置**设为 `-100`（PyTorch 的交叉熵会忽略 `-100`，这些位置不参与 loss）。
- 若有 `pad_token`，一般还要把 padding 位置的 `labels` 也设为 `-100`，避免对补齐符算损失。

`-100` 到底写在哪个向量里？（和 `input_ids` 的关系）

- **`input_ids`**：始终是「真实的 token id 序列」，**不会**被改成 `-100`。模型前向仍然按完整序列算 hidden states / logits；音频等其它字段也照常进 `batch`。
- **`labels`**：与 `input_ids` **同形状**（本项目里 `labels = full["input_ids"].clone()`），再在上面改数。传给 `model(**batch)` 时，字典里是 `input_ids=...` 和 **`labels=...` 两个键**。
- **`-100` 的含义**：在 **`labels` 张量**的对应位置上写入 `-100`，表示「这一格**不算**交叉熵损失」。不是「attention mask」里常见的 0/1，也不是改 logits；是 **Hugging Face `CausalLM` 约定：loss 里忽略 `label == -100` 的位置**。
- **叠加顺序（与本仓库一致）**：先在 `labels` 上把 **前缀** `[:, :pl]` 置 `-100`；再若有 `pad_token_id`，对 **`full["input_ids"] == pad_id`** 的位置把 `labels` 同样置 `-100`（`masked_fill`）。见 `musiccaps/lm_backend.py` 的 `_supervised_inputs`。

一句话：**mask 写在 `labels` 里；`input_ids` 保持可读 token；`-100` 是写在 `labels` 的某些时间步上，告诉 loss 别管这些步。**

注意表述：实现上是用 **prompt-only 序列的 token 长度 `pl`** 作为分界线，**不是**用「full 长度减 prompt 长度」再心算切分；`pl` 来自第二次构造，保证与 `processor.apply_chat_template` 的行为**完全一致**（包括角色标记、特殊 token 等）。

为什么不能只构造一次 full，自己切 assistant?

- 对话模板会在不同角色之间插入**特殊 token**；assistant 起点**不能**靠「数汉字」或「字符串 `find`」猜。
- 单独再 tokenize 一遍「无 assistant、且带 generation prompt」的 prompt，等价于让 **processor 自己声明**：在**当前模型与模板版本**下，前缀到底占多少个 token。这样 mask 不会错位；一旦错位，轻则学偏，重则 `labels` 全为 `-100`、loss 异常。

本项目代码对应关系（便于你对照仓库）：`musiccaps/lm_backend.py` 里 `_supervised_inputs` 先 `full = _apply_template(..., add_generation_prompt=False)`，再 `prompt = _apply_template(..., add_generation_prompt=True)`，取 `pl = prompt["input_ids"].shape[1]`，然后 `labels[:, :pl] = -100`。

5.2 功能

把“学习目标”精准聚焦在模型回答，而不是输入前缀。

5.3 公式（更完整版本）

设目标序列为 $y_1,\dots,y_T$，mask 集合为 M（只包含 assistant token 位置）：

$$ \mathcal{L} = - \frac{1}{|M|}\sum_{t \in M}\log p_\theta(y_t \mid x, y_{<t}) $$

这里 x 包含音频和 prompt 条件。

5.4 二级选项

- **A. assistant-only（本项目）**
  - 优点：目标干净，收敛稳定
  - 缺点：需要额外做 prompt 对齐
- **B. 全 token 都算**
  - 优点：实现简单
  - 缺点：优化方向偏移，容易学成“复读机”

5.5 替代方案

- selective span loss（只训关键片段）
  - 优点：更细粒度控制
  - 缺点：实现复杂，维护成本高

5.6 属性

- 复杂度：中
- 稳定性：高
- 成本：低（额外一次 prompt 构造）
- 可解释性：高

---

6. 为什么是 LoRA（而不是全参数微调）

6.1 LoRA 是什么

原模型权重 W 不直接改，学习一个低秩增量：

$$ W' = W + \Delta W,\quad \Delta W = BA,; \text{rank}=r $$

其中 A,B 是小矩阵，只训练它们。

6.2 功能

在较低显存和算力下，完成领域适配。

6.3 二级选项

- **A. LoRA（本项目）**
  - 优点：参数少、显存省、训练快、易保存 adapter
  - 缺点：容量有限，极端迁移上限可能低于全参
- **B. Full FT**
  - 优点：能力上限高
  - 缺点：成本高、遗忘风险高、迭代慢
- **C. QLoRA**
  - 优点：更省显存
  - 缺点：对量化和数值稳定更敏感
  - 人话展开见下文 **§6.8**

6.4 为什么这里优先 LoRA

- 任务是 caption 适配，不一定需要重塑全模型；
- 需要给后续 GRPO 延续训练，adapter 链路更自然；
- 对项目资源约束更友好。

6.5 属性

- 复杂度：中
- 稳定性：高（成熟方案）
- 成本：低
- 可迁移性：高（adapter 可独立管理）

6.6 SwiGLU 是什么（和 `gate_proj` / `up_proj` / `down_proj` 的关系）

很多现代 LLM（含 **Llama / Qwen** 系）的 FFN 不用「单层中间维 + 单一非线性」，而用 **SwiGLU**（Shazeer 提出的门控线性单元变体，常用 **SiLU** 作门控，文献里也叫 **SwiGLU** 当门控用 Swish/SiLU 时）。

对隐藏向量 x（单 token 的隐状态），一层里可写成示意：

$$ \text{FFN}(x) = W_{\text{down}}\Bigl(\sigma(x W_{\text{gate}}) \odot (x W_{\text{up}})\Bigr) $$

- σ 常为 **SiLU**（即 Swish 一族）
- $\odot$ 为按元素乘
- **三个线性层**在代码里通常就叫 `gate_proj`、`up_proj`、`down_proj`（名字因实现略有出入，但 Qwen/Llama 风格一致）

和旧式 FFN 对比（面试一句）：老式可能是 `W_1` → 激活 → `W_2`；SwiGLU 多了一条 **gate** 支路，表达更灵活，参数量略增，在很多模型上效果更好。
这也解释了：为什么本项目的 LoRA 目标里会同时出现 **三个 FFN 投影名**，而不是只有一个 `fc1/fc2`。

6.7 本仓库 LoRA 打在模型的哪些参数上

LoRA **不是**整模更新：由 `lora_target_modules` 指定子模块**名字**，`peft` 在**名称匹配的线性层**上注入 $\Delta W = BA$，其余权重 `requires_grad=False`（冻结）。

默认配置（Qwen2.5-Omni Thinker 路径，`OmniThinkerBackend`）
与 `configs/default.yaml`、`TrainConfig` 一致，默认为：

`target_modules`	含义（每层 Transformer block 内）
`q_proj`, `k_proj`, `v_proj`, `o_proj`	自注意力：**Q / K / V / O** 四个线性投影
`gate_proj`, `up_proj`, `down_proj`	**SwiGLU FFN** 的三条投影（见 §6.6）

默认通常不包含：词嵌入（如 `embed_tokens`）、输出头（`lm_head`）、**LayerNorm / RMSNorm** 等——除非你把对应模块名显式加进列表且 PEFT 能匹配到。

多模态（Omni）注意：音频编码器等子模块若**没有**与列表同名的层，则**不会**被注入 LoRA；实际「有没有训到音频侧」取决于 Thinker 里**模块命名**是否与上述字符串匹配。若要加训某条支路，需在模型里确认 `named_modules()` 中的名字后，再扩展 `lora_target_modules`。

Debug 路径（Tiny GPT-2 mock）：代码里写死为 `c_attn`、`c_proj`（GPT-2 风格命名），与 Omni 默认列表不同。

改范围：编辑配置里的 `lora_target_modules` 即可增删 LoRA 挂载层；保存的 adapter 与训练时的 `LoraConfig` 需一致（见 `GRPO_METHOD.md` 里关于 `adapter_config` 的说明）。

面试一句话：默认 LoRA 只训 **decoder 里每层的注意力四投影 + SwiGLU 三投影**；具体以配置列表为准，多模态支路是否覆盖要看模块名是否匹配。

6.8 QLoRA 在干什么（读懂版）

可以记三层：

1. **底座权重「压缩住」**：大部分预训练参数用 **4-bit（常见 NF4）** 等形式放在显存里，**同样大小的模型占的显存远小于 bf16 全存**。这不是把数学换成别的任务，只是**存与读的表示更省位宽**。

2. **前向时仍要算矩阵乘**：实现上会在计算时把用到的权重 **反量化到更高精度**（如 bf16）再做乘法；**梯度主要流向 LoRA 那两条小矩阵** A,B，而不是去存一份完整的 fp32 底座优化器状态。
3. **LoRA 照旧**：在选定的线性层上仍是 $\Delta W \approx BA$，**可训练参数很少**；优化器状态（Adam 的动量等）主要跟这些小块走，所以 **「大模型 + 单卡」** 才变得可行。

和「普通 LoRA」差在哪？ 普通 LoRA：底座往往是 bf16/fp16 **全精度驻显存**。QLoRA：**底座 4-bit 省显存**，换回来的是对 **量化方案、反量化、数值范围** 更敏感，超参要多盯一眼。

一句背：QLoRA = **4-bit 底座省显存 + 上面挂正常 LoRA 训小适配器**，优化器不扛全量 fp32 底座。

6.9 LoRA 合并（merge）常见坑

推理时把 LoRA 融进底层权重：$W' = W + \alpha/r \cdot BA$（具体缩放以 `lora_alpha` 与实现为准），得到「普通 `nn.Linear`」，省掉旁路计算。常见坑：

- **不可逆**：合并后难以再拆回「纯底座 + adapter」；若还要 **继续训 LoRA、换 rank、多适配器切换**，应保留 **未合并** 的 PEFT 模型或单独存 adapter。
- **dtype / 设备**：合并要在 **与训练一致的 dtype 思路**下做（如 bf16）；混精度下若手工合并，注意 W 与 ΔW **同一类型再相加**，避免静默精度损失。
- **多适配器**：`merge` 通常针对**当前激活**的一套；多个 adapter 时要弄清 **先加哪套、是否逐套 merge**，否则权重与预期不一致。
- **与加载端契约**：合并后的 `state_dict` **结构变了**（不再有 `lora_A`/`lora_B`），部署脚本若写死 PEFT 加载会失败；需按 **融合后的普通模型** 保存与加载。
- **rank / target_modules 不一致**：磁盘上的 adapter 与当前模型 `LoraConfig` 不一致时，合并或加载会报错或 **silent wrong**（见 `GRPO_METHOD.md` 中 adapter 配置一致性的说明）。

---

7. 优化器与训练更新（AdamW + 反向传播）

7.1 定义

训练核心三步：

1. 前向算 loss
2. 反向算梯度
3. 优化器更新参数

7.2 AdamW 在做什么（人话）

- Adam：给每个参数自适应学习率（根据历史梯度一阶/二阶统计）
- W：把权重衰减和梯度更新解耦，正则化更干净

7.3 为什么不是 SGD

- 大模型 + 稀疏高维场景里，SGD 对学习率更敏感，收敛慢；
- AdamW 在 Transformer 训练上经验更成熟，启动成本低。

7.4 梯度裁剪是什么

如果某一步梯度太大，就按比例缩小到阈值（例如 1.0）。
作用是防止梯度爆炸导致 loss NaN 或训练发散。

7.5 属性

- 复杂度：低
- 稳定性：高
- 成本：中
- 可解释性：中高

---

8. 数值精度（bf16/fp16/fp32）

8.1 定义

`dtype` 是参数和激活的数值表示精度。

8.2 为什么常用 bf16

- 比 fp16 动态范围更大，不容易溢出；
- 比 fp32 更省显存、更快。

8.3 二级选项

- bf16：速度和稳定的折中（本项目默认）
- fp16：更快但更脆
- fp32：最稳但最慢最占显存

8.4 属性

- 复杂度：低
- 稳定性：bf16 高，fp16 中，fp32 最高
- 成本：fp32 最高

---

9. 为什么先 SFT 再 GRPO

9.1 定义

- SFT：先学“基本会答”
- GRPO：再学“答得更符合奖励目标”（如 aspect/CLAP）

9.2 功能

SFT 提供可用初始策略，降低 RL 直接起步的不稳定性。

9.3 为什么不直接 RL

- 直接 RL 早期探索噪声大、训练不稳、样本效率低；
- SFT 先把语言能力和任务格式对齐，再做偏好优化更稳。

9.4 属性

- 复杂度：中（两阶段）

- 稳定性：高于直接 RL
- 成本：中

---

10. 每个关键超参数，到底在控制什么

10.1 `sft_learning_rate`

- 控制“每一步改多大”
- 太大：震荡/发散
- 太小：收敛慢/欠拟合
- 通常是最优先调的参数

10.2 `sft_batch_size`

- 控制每步统计多少样本
- 大 batch：梯度更稳，显存更高
- 小 batch：噪声更大，但便宜

10.3 `lora_r`

- 控制 LoRA 容量（rank 越大，可表达更新越丰富）
- 太小：学不动
- 太大：成本高，过拟合风险增加

10.4 `lora_dropout`

- 给 LoRA 分支做正则，防止过拟合
- 太高会欠拟合

10.5 `sft_max_grad_norm`

- 梯度安全阈值
- 太小可能“刹车过猛”，太大保护不够

10.6 `lora_target_modules`

- 控制 **哪些子模块**挂 LoRA（按**模块名字符串**匹配，见 **§6.7**）
- 默认覆盖 **注意力四投影 + SwiGLU 三投影**；改列表即改变「可训参数」集合与 adapter 结构

---

11. 二级选项与替代方案速查表（面试快答）

11.1 损失函数

- 主选：token-level cross entropy
- 替代：label smoothing CE、focal loss（长尾场景）、RL reward loss（后训练）

11.2 参数高效微调

速查：主选 **LoRA**；替代 **Full FT**、**QLoRA**、**Adapter Tuning**、**Prefix Tuning**。LoRA 的公式与「为何本项目优先」见上文 **§6**；本节侧重对比与面试口述。

LoRA（主选）

- **是什么**：在选定线性层上学习低秩增量 $\Delta W = BA$，**冻结**原 W，只更新 A,B（及可选 bias）。推理时可把 ΔW **合并进** W' 做常规模型前向，不增加推理延迟；合并坑见 **§6.9**。
- **优点**：可训练参数少、显存友好、checkpoint 小、多任务可存多套 adapter；生态成熟（PEFT、vLLM 等）。
- **缺点**：秩 r 有容量上限；极强分布外重塑时可能不如全参。
- **何时选**：资源受限、要快速迭代、要接后续阶段（如 GRPO）只换训练头时的默认方案。

Full FT（全参数微调）

- **是什么**：对模型全部权重（或绝大部分）求梯度并更新。
- **优点**：表达容量最大，没有「只动旁路」的结构限制；数据极大、任务与预训练差很远时上限更高。
- **缺点**：显存与算力高；checkpoint 大；**灾难性遗忘**与稳定性风险更大；实验迭代慢。
- **何时选**：算力充足、必须重塑表征、或 LoRA/QLoRA 反复不够时再考虑。

QLoRA

- **是什么**：**底座权重以低比特（如 4-bit NF4）驻显存**，在其上仍挂 **LoRA** 等高精度适配器；优化器状态主要落在可训练的小参数上。
- **人话**：显存大头给「压扁的底座」，真正梯度更新集中在 LoRA 小块；详见 **§6.8**。
- **优点**：同等 GPU 上能塞更大的 base model；单卡训 7B/13B 类模型的常用手段。
- **缺点**：依赖量化质量与实现；数值与超参（如 double quant、分页优化器）更敏感；调试难度略高于纯 LoRA。
- **何时选**：显存是硬约束、必须把大模型拉起来训，且能接受多调一轮稳定性时。

Adapter Tuning

- **是什么**：在 Transformer 子层（如 FFN）旁路插入小的 **瓶颈 MLP（down-project → 非线性 → up-project）**，原层输出与 adapter 输出相加；**仅 adapter 参数可训**（经典 Adapter / Parallel Adapter 等变体）。
- **和 U-Net 无关**：**不是**图像里的 U-Net。U-Net 是 **编码器–解码器 + 跳连** 的整网结构，用于分割/扩散等；Adapter 只是在 Transformer 里加了一截 **窄一点的 FFN 旁路**，没有对称解码器，也没有 U 形多尺度语义。**「瓶颈」指的是中间维比隐藏维小，不是 U-Net 架构。**
- **优点**：模块化强，可多任务叠不同 adapter；不动原权重，与「插件式」部署思路一致。
- **缺点**：默认实现会多一段前向计算，**延迟与吞吐**通常差于合并后的 LoRA；层数多时节流参数与插入位置要设计。
- **何时选**：强多任务隔离、需要频繁切换任务头而不想动基底权重时。

Prefix Tuning（及相近的 soft prompt）

- **是什么**：在注意力中引入**可训练的前缀**：早期做法是给每层 K/V 前拼接一段可学习的「虚拟 token」表征；与 **Prompt Tuning**（仅输入侧连续向量）同属「训提示、不训主体权重」一族。
- **优点**：可训练参数量可以极少；不动 backbone 矩阵，适合超大模型上「只调前面几兆参数」的探索。
- **缺点**：前缀占用序列长度与 KV 预算；与具体模板、长度强相关，迁移与维护有时不如 LoRA 直观；部分场景下收敛与最终效果依赖任务。
- **何时选**：冻结超大底座、只允许极小参数预算，或研究与 prompt 空间本身相关的课题时。

对比一句话（背）：要 **省参数 + 工程默认** → LoRA；要 **极限容量** → Full FT；要 **单卡大模型** → QLoRA；要 **多任务插件** → Adapter；要 **极少可训参数、动提示不动层** → Prefix / Prompt Tuning。

11.3 数据组织

- 主选：manifest jsonl
- 替代：Parquet/数据库/在线样本服务

11.4 训练稳定性

- 主选：AdamW + grad clip + seed
- 替代：Adafactor、EMA、更强监控和早停策略

---

12. 属性评分模板（答“工程能力题”时可直接用）

你可以按 1~5 分快速比较方案：

- **复杂度**：开发和维护难度
- **稳定性**：是否容易发散、出错
- **算力成本**：显存、时长、硬件要求
- **可解释性**：问题是否容易定位
- **扩展性**：后续接 GRPO 或新任务是否顺滑

示例（本项目方案）：

- assistant-only CE：复杂度 3，稳定性 5，可解释性 5
- LoRA：复杂度 3，稳定性 4，成本 5，扩展性 5
- 固定模板：复杂度 2，稳定性 5，扩展性 3

---

13. 高频追问（详细版）

Q1：交叉熵到底在“惩罚”什么？

惩罚的是：模型把真实 token 赋予太低概率。
真实 token 概率越低，$-\log p$ 越大，损失越大。

Q2：为什么是 next-token 预测，而不是整句一次性预测？

因为自回归语言模型的生成机制就是一步一步预测下一个 token。
训练目标和推理机制一致，工程最稳。

Q3：为什么要保存 adapter，不直接存整模？

因为我们训练的是增量参数，adapter 更轻、更易管理，也方便后续阶段叠加或切换。

Q4：如何判断 SFT 学到了“听音频”而不是只学模板？

看对不同音频的输出是否有内容差异，并结合抽样听感和定量指标（如 aspect 覆盖）验证。

---

14. 90 秒口述稿（可直接背）

这个项目的 SFT 本质是条件生成：条件是音频和固定指令，目标是 MusicCaps 的 caption。输入被组织成 system、user、assistant 三段，其中 assistant 是要学习的答案。模型训练时按 token 做 next-token 预测，我们用交叉熵来度量“模型给真实 token 的概率够不够高”。为了让目标干净，只在 assistant token 上算损失，system 和 user 部分用 mask 忽略。参数更新上我们采用 LoRA，不改全模型，只训练低秩增量参数，换来更低显存和更快迭代，并且便于保存 adapter 接到后续 GRPO。优化器使用 AdamW，加梯度裁剪和固定随机种子来保证稳定性。整体设计思路是先把基础描述能力训稳，再进入奖励驱动的后训练阶段。

---

15. 可直接放简历的表述

- Designed an audio-conditioned SFT pipeline with assistant-only cross-entropy via prompt masking.
- Implemented PEFT-based adaptation (LoRA) for multimodal caption generation under constrained GPU budgets.
- Built a manifest-driven, reproducible training workflow with stability controls (AdamW, grad clipping, seed management).